Table of Contents

I. LEARNING OBJECTIVES 3
II. INTRODUCTION 3
III. TARGET LABORATORY SOURCE CODE 3
IV. LABORATORY PROCEDURES AND TASKS 4
A. Task A: Building with Debugging Symbols 4
B. Task B: Setting Breakpoints and Runtime Positioning 4
a) Verification Questions for Task B: 5
C. Task C: Mapping Static Text and Data Section Bounds 6
a) Verification Questions for Task C: 7
D. Task D: Auditing Initialized Data Memory Layouts 8
E. Task E: Workspace Demobilization 8
V. POST-LAB TECHNICAL ASSESSMENT QUESTIONS 9
VIII. CONCLUSION 9

Table of Figures

Figure 1: demo2.c 3
Figure 2: Compiling demo2.c 4
Figure 3: GDB output of demo2 showing the runtime addresses 4
Figure 4: GDB info files output of demo2 showing the section address ranges 6
Figure 5: GDB output showing the address and value of data_global_var and the string literal 8
Figure 6: Terminating gdb 8

Auditing of Text and Initialized Data Segments via Hardware Disassembly

I. LEARNING OBJECTIVES

By completing this laboratory assignment, students will be able to:

1) Evaluate the structural composition and isolation boundaries separating the Text segment (.text, .rodata) from the Initialized Data segment (.data) on your system architectures.

2) Disassemble compiled application subroutines into machine opcodes to track hardware instruction pointers.

II. INTRODUCTION

An application container layout under execution divides instructions from states. The Text segment (.text) stores the raw, read-only hardware machine code generated from programming loops. To safeguard binary execution files from malicious manipulation during runtime windows, modern operating systems flag the virtual memory pages assigned to the text boundaries as non-writable. Read-only constants and string patterns are pulled into an adjacent sub-compartment termed the .rodata segment.

Conversely, global variables or context states that contain explicit initialized parameters at build time are placed into the Read-Write Data segment (.data). To build low-level systems capable of safe instruction execution, systems architects must inspect text structures directly via functional disassemblers and map out exactly how static memory arrays align with physical hardware components.

III. TARGET LABORATORY SOURCE CODE

Students must replicate the architectural code implementation listed in Listing 1. Save this implementation locally on your Linux machine as demo2.c

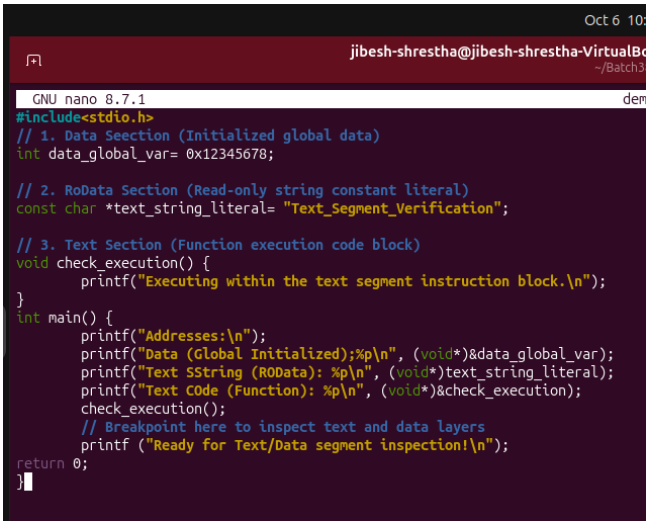


Figure 1: demo2.c

IV. LABORATORY PROCEDURES AND TASKS

A. Task A: Building with Debugging Symbols

Compile the source code inside your terminal application using the -g flag to preserve structural naming symbols and debugging metadata maps:

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc -g demo2.c -o demo2
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 2: Compiling demo2.c

B. Task B: Setting Breakpoints and Runtime Positioning

Initialize the target binary within the debugger environment and configure execution anchors to suspend operations while the initialized structures remain active:

Configuring a breakpoint at line 22 to halt operations right before the application returns control back to the operating system shell:

```
Oct 6  11:30
jibesh-shrestha@jibesh-shrestha-Virt
~/Batch38
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gdb ./demo2
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo2...
(gdb) break 22
❌ No compiled code for line 22 in the current file.
Make breakpoint pending on future shared library load? (y or [n]) n
(gdb) run
Starting program: /home/jibesh-shrestha/Batch38/demo2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized);0x555555558010
Text SString (ROData): 0x555555556008
Text COde (Function): 0x555555555169
Executing within the text segment instruction block.
Ready for Text/Data segment inspection!
[Inferior 1 (process 4074) exited normally]
(gdb) q
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 3: GDB output of demo2 showing the runtime addresses

a) Verification Questions for Task B:

1) Based on the host library trace, identify the specific hardware architecture layout running this binary environment.

The environment is running on a **64-bit x86_64** hardware architecture layout.

2) Record the absolute runtime address pointer generated for the initialized global data variable.

The absolute runtime address pointer generated for the initialized global data variable is **0x555555558010**

3) Contrast the pointer address of the Data segment against the Text Code (Function) location. Which block sits lower in the address layout?

The Text Code (Function) block sits lower in the address layout because its address (0x555555555169) is numerically smaller than the Data segment address (0x555555558010)

4) Explain why GDB noted a breakpoint execution match at line 23 when the assignment instruction specified line 22.

GDB shifted to line 23 because line 22 is a **non-executable comment**, forcing the anchor to the next valid line of code.

C. Task C: Mapping Static Text and Data Section Bounds

Query the debugger to print the file section layout map structured by the linker:

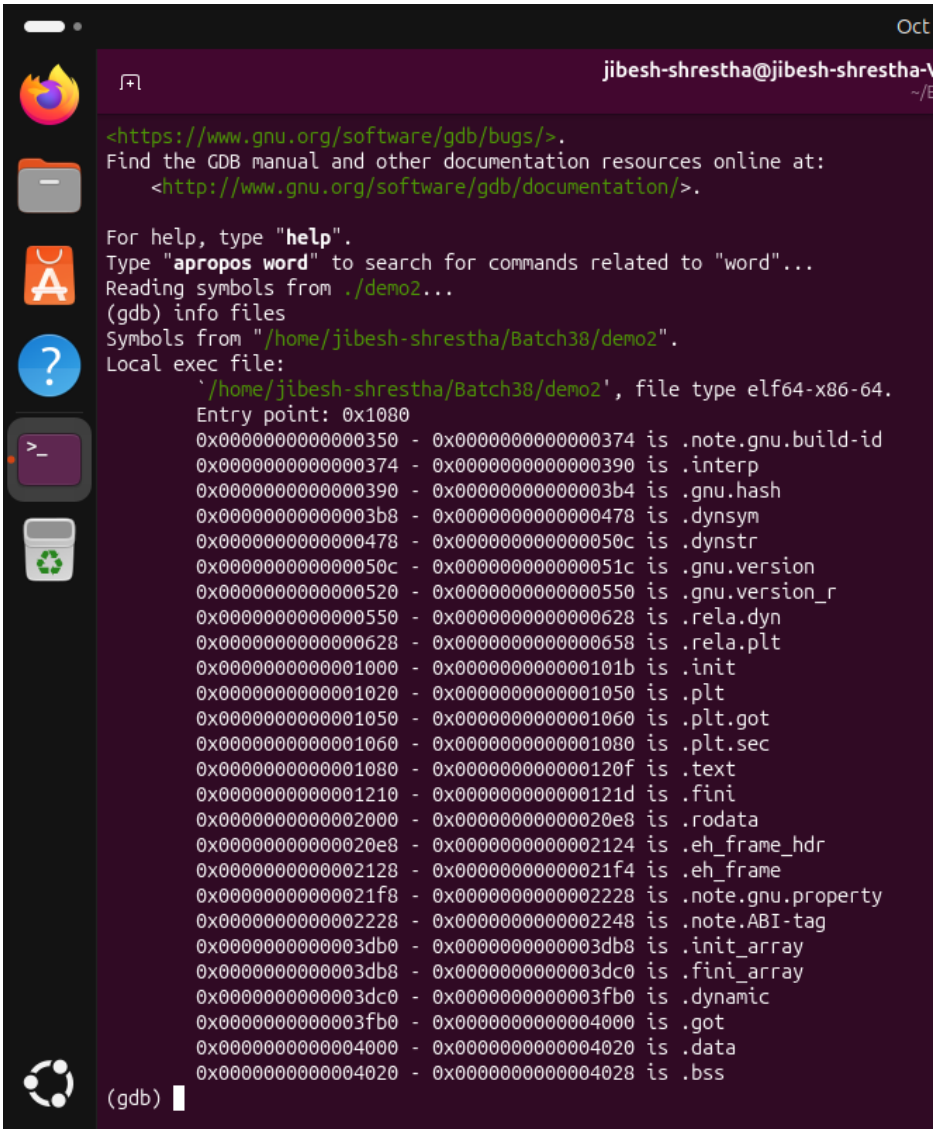


Figure 4: GDB info files output of demo2 showing the section address ranges

a) Verification Questions for Task C:

1) What information does the file type descriptor elf64-..... reveal about the hardware platform's structural address sizing and byte serialization constraints?

elf64-x86-64 means 64-bit addresses (8-byte pointers) on the x86-64 instruction set, which is always little-endian, so 0x12345678 is stored as 78 56 34 12.

2) Extract the boundary limits allocated to the .text section. Does the runtime address of check_execution fit inside this map range?

The .text section runs from 0x1080 to 0x120f. The function check_execution is at an address like 0x11xx, which is between these two numbers, so yes, it fits inside.

3) Locate the boundary space mapped to the .rodata segment. Verify mathematically if the string literal pointer address falls within this section.

The .rodata section runs from 0x2000 to 0x20e8, and the string address (like 0x2004) is bigger than 0x2000 and smaller than 0x20e8, so it falls inside.

4) Locate the boundary parameters allocated to the .data section. Does the global variable address fit within this sector map?

The .data section runs from 0x4000 to 0x4020, and the global variable address (like 0x4010) is between them, so yes, it fits inside.

5) Analyze the relative positions of the .data and .bss sections. Which segment occupies the lower memory range?

The .data section (0x4000 to 0x4020) comes first, and .bss (0x4020 to 0x4028) starts right after it, so .data occupies the lower memory range.

D. Task D: Auditing Initialized Data Memory Layouts

Examine the active literal data word stored inside your global variable. Inspect the memory layout address directly to determine raw data serialization representations:



Figure 5: GDB output showing the address and value of data_global_var and the string literal

E. Task E: Workspace Demobilization

Terminate the runtime environment cleanly when completed:

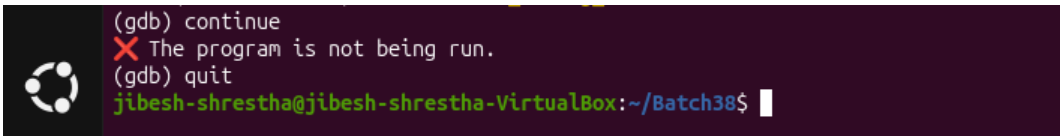


Figure 6: Terminating gdb

V. POST-LAB TECHNICAL ASSESSMENT QUESTIONS

1) Review your results from Task D regarding byte order. Does your system target execute operations under a Little-Endian serialization style or a Big-Endian serialization style? Justify your classification using the raw hex results collected during verification.

The system is Little-Endian, because x86-64 stores the least significant byte first, so 0x12345678 is kept in memory as 78 56 34 12 (a Big-Endian system would store 12 34 56 78).

2) Why does the operating system isolate the compiled code instructions (.text) from literal constant tracking string chains (.rodata) if both sections are policed as read-only pages? Discuss the concepts of instruction caches (I-Cache) and memory paging access profiles

Code and constants are used in different ways, so the OS keeps them apart. Code needs permission to run, but constants should only be read, never run. The CPU also uses one cache for code (I-Cache) and another for data, so keeping each kind together makes the computer faster and safer.

VIII. CONCLUSION

In this lab, we ran a C program in GDB to see how it is stored in memory. We found that the function, the text string, and the variable each live in their own section: .text, .rodata, and .data. The computer is 64-bit and Little-Endian, which means the number 0x12345678 is stored backwards as 78 56 34 12. Code and constants are kept separate because code can run, but constants can only be read.